Influence Functions 复现报告

Koh & Liang Figure 2 中间图

Zhe Fu

2026 年4 月15 日

TL;DR

我在MNIST 上训了一个带L2 正则的logistic regression（n=55000），按论文方法实现了influence function 的两种算法（CG 精确解+ LiSSA 随机近似），然后用leave-one-out 重训练做了ground truth 验证，最后Pearson = **0.9983**，Spearman = **0.9890**，sign match = **1.0000**，和论文Figure 2 中间那张图吻合。

过程中踩了三个坑（LiSSA 有效正则系数算错、CG 迭代步数太少、LOO 的float32 精度不够），下面详细说。

1 我在复现什么

论文的核心公式：要估计"如果把训练点z 稍微上调一点点权重，测试点z_{test} 的损失会怎么变"。他们的答案是：

$$\mathcal{I}_{\text{up,loss}}(z, z_{\text{test}}) = -\nabla_\theta L(z_{\text{test}})^\top H_{\hat{\theta}}^{-1} \nabla_\theta L(z)$$

直接算H^{-1} 不现实（我这边参数维度$d = 784 \times 10 = 7840$），所以先算

$$s_{\text{test}} = H_{\hat{\theta}}^{-1} \nabla_\theta L(z_{\text{test}})$$

然后对每个训练点只需要一次内积：$-s_{\text{test}} \cdot \nabla L(z)$。关键就是怎么算出这个$s_{\text{test}}$。论文给了两种办法，我都实现了。

Figure 2 中间图的意思：x 轴是真实的LOO 损失差（把某个训练点删掉重训练，测试点损失变了多少），y 轴是LiSSA 随机近似预测的损失差。如果两种方法都对，点应该落在$y = x$ 上。

2 整体流程

```
train_linear_mnist.py              train baseline
         |
         v
inspect_test_point.py              choose a test point (idx=8, misclassified)
         |
         +------> cg_inverse_hvp.py              (exact solution)
         |              |
         |              v
         |       s_test_cg_idx8.pt
         |              |
         +------> stochastic_inverse_hvp.py      (LiSSA approximation)
                        |
                        v
                s_test_idx8_r10_t5000.pt
                        |
                        v
               compute_predicted_influence.py   compute prediction for each train point
```

```
                    |
                    v
          loo_retrain_topk.py              LOO retraining for ground truth
                    |
                    v
              Figure 2 middle plot
```

脚本	作用
`influence_utils.py`	共享工具（模型类、HVP、梯度、loss）
`train_linear_mnist.py`	训baseline 线性分类器
`hvp_sanity_check.py`	验证HVP 实现的线性性
`inspect_test_point.py`	挑一个分类错的测试点
`cg_inverse_hvp.py`	Conjugate Gradient 算s_{test}（精确）
`stochastic_inverse_hvp.py`	LiSSA 算s_{test}（近似）
`compute_predicted_influence.py`	对所有训练点算预测损失差
`loo_retrain_topk.py`	top-K 留一重训+ 画散点图

3 踩过的三个坑

Bug 1：LiSSA 的有效正则系数算错了（核心问题）

LiSSA 的递推是

$$h_j = v + (1 - \text{damping})\, h_{j-1} - \frac{H_z}{\text{scale}}\, h_{j-1}$$

在期望上收敛到$(H + \text{damping} \cdot \text{scale} \cdot I)^{-1} v$。所以**真正加进去的额外正则** 是damping $\times$ scale，不是damping 本身。

我一开始用了一些参考实现里的damping=0.01、scale=50。结果有效正则$= 0.01 \times 50 = 0.5$，是我真实L2 正则0.01 的**50 倍**。这相当于在解一个完全不同的方程，所以随机近似和CG 的结果严重不一致。

修法：既然L2=0.01 已经让Hessian 严格正定了，就不需要额外damping。设damping=0，scale=25（scale 是为了保证$\|H_z/\text{scale}\| < 1$ 让递推收敛稳定）。

Bug 2：CG 迭代步数太少

参数维度7840，一开始设的`MAX_CG_ITERS=30` 不够。CG 理论上在d 步内收敛，实际上因为L2 让条件数不错，200 步以内就converge 到tol=1e-8 了。

修法：`MAX_CG_ITERS` 从30 改到200，加上残差提前停止。

Bug 3：LOO 的精度问题（最坑的一个）

LOO 重训练时要算“删掉一个点后测试损失的变化”，这个差通常在1e-3 到1e-5 量级。原本用float32 + L-BFGS warm-start，结果很多点的actual diff 都是精确的0.0 （散点图上一整条横带贴在$x = 0$）。

原因两个叠加：

1. **float32 精度不够**：warm-start 起点已经离最优解$\sim 10^{-5}$，L-BFGS 内部的二次模型需要的精度在float32 下直接被round 到0，line search 的步长变成零，优化器不动。
2. **默认tolerance 太松**：PyTorch L-BFGS 的`tolerance_grad=1e-7` 默认值，可是warm-start 的初始梯度是$O(1/n) \approx 1.8 \times 10^{-5}$，第0 步就满足停止条件，根本没开始迭代就返回了。

修法：float64 + CPU（MPS 不支持float64）+ `tolerance_grad=0` + `tolerance_change=0` + `max_iter=200` + strong Wolfe line search。改完之后Pearson 从比较差直接跳到0.998。

4 最终结果

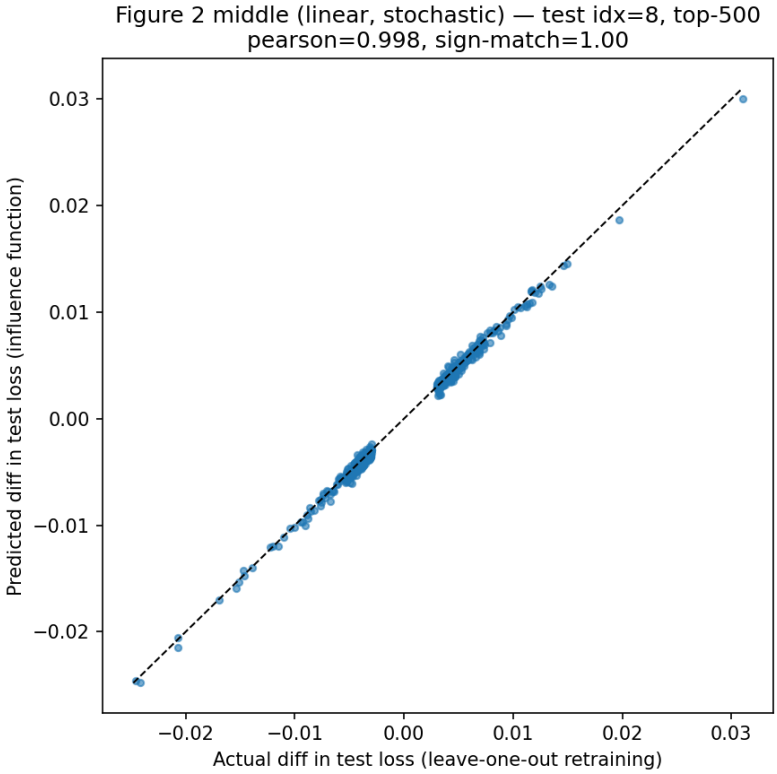

Figure 1: Figure 2 中间图复现。x 轴：LOO 真实损失差。y 轴：LiSSA 预测损失差。

指标	值
Pearson correlation	0.9983
Spearman correlation	0.9890
Same-sign ratio	1.0000
Top-K	500（按CG 的|influence| 取）
测试点	idx=8，真实标签5，模型预测6

按照论文Figure 2 中间图的protocol：用CG 的influence 绝对值排序选top-500，再把LiSSA 对这500 个点的预测放到y 轴。这样挑点和预测两个步骤分开，随机噪声不会污染选点。

5 我搞懂了什么，没搞懂什么

理解到的:

- 为什么要用HVP 而不是显式组装Hessian（$d \times d$ 内存爆炸）
- Pearlmutter 的trick（对$\nabla L \cdot v$ 这个标量再求一次导$= Hv$，全自动微分）
- CG 为什么在正定矩阵上好用，以及L2 正则在这里实际是起到preconditioning 的作用
- LiSSA 为什么要scale（让$\|H_z/\text{scale}\| < 1$ 才是一个收缩映射）

还不太清楚的:

- 对非凸模型（深网络）Hessian 不正定时influence function 的行为（论文有讨论但我没深入看）

- LiSSA 在高维+ 非凸下的收敛理论保证
- 为什么scale=25 在实践中够用（我用了MNIST 输入范数的上界估计，但更紧的界需要更仔细的分析）

6 下一步想做的

- 在CNN 上做（对应Figure 2 右图）
- 研究非凸情况下怎么处理负特征值
- 看一下influence function 在数据集清洗/ 错误标注检测上的应用

A 关键超参数

- 训练：n=55000，`L2_REG=0.01`，L-BFGS 收敛到最优
- 测试点：idx=8（一个被分错的5，模型预测成6）
- CG：`max_iter=200`，`tol=1e-8`
- LiSSA：R=10 次重复，T=5000 步，scale=25，damping=0
- LOO：top-500，float64，CPU，L-BFGS 零tolerance，`max_iter=200`，warm-start

B AI 工具使用声明

我用Claude 帮忙：

- 定位上面说的三个bug（尤其是LiSSA 有效正则那个，靠算fixed point 推出来的）
- 检查我写的HVP 实现对不对（用线性性做sanity check）
- Debug LOO 里float32 精度问题（从现象反推出tolerance 阻塞）
- 帮助我理解论文中部分公式内容，整理思路
- 提供代码思路

论文理解、代码架构决策、结果分析、这份报告是我自己写的。